Welcome to Codingbat. See help for the latest.

Java | Python

Warmup-1 ☆☆☆☆
Simple warmup problems to get started, no loops (solutions available)
✓✓✓✓✓✓✓✓✓✓✓

Warmup-2 ☆☆☆
Medium warmup string/list problems with loops (solutions available)
✓✓✓✓✓✓✓✓✓

String-1 ☆☆☆☆
Basic python string problems -- no loops
✓✓✓✓✓✓✓✓✓✓✓

List-1 ☆☆☆☆
Basic python list problems -- no loops.
✓✓✓✓✓✓✓✓✓✓✓✓

Logic-1 ☆☆☆
Basic boolean logic puzzles -- if else and or not
✓✓✓✓✓✓✓✓✓

Logic-2 ☆☆☆
Medium boolean logic puzzles -- if else and or not
✓✓✓✓✓✓✓

String-2 ☆☆
Medium python string problems -- 1 loop.
✓✓✓✓✓✓

List-2 ☆☆
Medium python list problems -- 1 loop.
✓✓✓✓✓✓

iii. Warmup-1

Warmup-1 > sleep_in
prev | next | chance

The parameter weekday is True if it is a weekday, and the parameter vacation is True if we are on vacation. We sleep in if it is not a weekday or we're on vacation. Return True if we sleep in.

sleep_in(False, False) → True
sleep_in(True, False) → False
sleep_in(False, True) → True

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def sleep_in(weekday, vacation):
  if not weekday or vacation:
    return True
  else:
    return False
```

Go

Expected	Run	
sleep_in(False, False) → True	True	OK
sleep_in(True, False) → False	False	OK
sleep_in(False, True) → True	True	OK
sleep_in(True, True) → True	True	OK

✓ All Correct

next | chance

Python > Warmup-1

l_maratkyzy@kbtu.kz done page

Your progress graph for this problem

Warmup-1 > diff21

prev | next | chance

Given an int n, return the absolute difference between n and 21, except return double the absolute difference if n is over 21.

diff21(19) → 2
diff21(10) → 11
diff21(21) → 0

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def diff21(n):
  if n < 21:
    d = abs(21-n)
  else:
    d= abs((21-n)*2)
  return d
```

Expected	Run		
diff21(19) → 2	2	OK	
diff21(10) → 11	11	OK	
diff21(21) → 0	0	OK	
diff21(22) → 2	2	OK	
diff21(25) → 8	8	OK	
diff21(30) → 18	18	OK	
diff21(0) → 21	21	OK	
diff21(1) → 20	20	OK	
diff21(2) → 19	19	OK	
diff21(-1) → 22	22	OK	
diff21(-2) → 23	23	OK	
diff21(50) → 58	58	OK	

All Correct

next | chance

Warmup-1 > front3

prev | next | chance

Given a string, we'll say that the front is the first 3 chars of the string. If the string length is less than 3, the front is whatever is there. Return a new string which is 3 copies of the front.

front3('Java') → 'JavJavJav'
front3('Chocolate') → 'ChoChoCho'
front3('abc') → 'abcabcabc'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def front3(str):
  front_end = 3
  if len(str) < front_end:
    front_end = len(str)
  front = str[:front_end]
  return front + front + front
```

Expected	Run		
front3('Java') → 'JavJavJav'	'JavJavJav'	OK	
front3('Chocolate') → 'ChoChoCho'	'ChoChoCho'	OK	
front3('abc') → 'abcabcabc'	'abcabcabc'	OK	
front3('abcXYZ') → 'abcabcabc'	'abcabcabc'	OK	
front3('ab') → 'ababab'	'ababab'	OK	
front3('a') → 'aaa'	'aaa'	OK	
front3('') → ''	''	OK	

All Correct

next | chance

Python > Warmup-1

l_maratkyzy@kbtu.kz done page

Your progress graph for this problem

Warmup-1 > not_string

prev | next | chance

Given a string, return a new string where "not " has been added to the front. However, if the string already begins with "not", return the string unchanged.

not_string('candy') → 'not candy'
not_string('x') → 'not x'
not_string('not bad') → 'not bad'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def not_string(str):
  if len(str) >= 3 and str[:3] == "not":
    return str
  return "not " + str
```

Expected	Run		
not_string('candy') → 'not candy'	'not candy'	OK	
not_string('x') → 'not x'	'not x'	OK	
not_string('not bad') → 'not bad'	'not bad'	OK	
not_string('bad') → 'not bad'	'not bad'	OK	
not_string('not') → 'not'	'not'	OK	
not_string('is not') → 'not is not'	'not is not'	OK	
not_string('no') → 'not no'	'not no'	OK	

All Correct

next | chance

Python > Warmup-1

l_maratkyzy@kbtu.kz done page

Your progress graph for this problem

Warmup-1 > makes10

prev | next | chance

Given 2 ints, a and b, return True if one if them is 10 or if their sum is 10.

makes10(9, 10) → True
makes10(9, 9) → False
makes10(1, 9) → True

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def makes10(a, b):
  return (a == 10 or b == 10 or a+b == 10)
```

Expected	Run		
makes10(9, 10) → True	True	OK	
makes10(9, 9) → False	False	OK	
makes10(1, 9) → True	True	OK	
makes10(10, 1) → True	True	OK	
makes10(10, 10) → True	True	OK	
makes10(8, 2) → True	True	OK	
makes10(8, 3) → False	False	OK	
makes10(10, 42) → True	True	OK	
makes10(12, -2) → True	True	OK	

All Correct

next | chance

Python > Warmup-1

l_maratkyzy@kbtu.kz done page

Your progress graph for this problem

Warmup-1 > sum_double

prev | next | chance

Given two int values, return their sum. Unless the two values are the same, then return double their sum.

sum_double(1, 2) → 3
sum_double(3, 2) → 5
sum_double(2, 2) → 8

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def sum_double(a, b):
  sum = a + b

  if a == b:
    sum = sum * 2
  return sum
```

Expected	Run		
sum_double(1, 2) → 3	3	OK	
sum_double(3, 2) → 5	5	OK	
sum_double(2, 2) → 8	8	OK	
sum_double(-1, 0) → -1	-1	OK	
sum_double(3, 3) → 12	12	OK	
sum_double(0, 0) → 0	0	OK	
sum_double(0, 1) → 1	1	OK	
sum_double(3, 4) → 7	7	OK	

All Correct

next | chance

Python > Warmup-1

l_maratkyzy@kbtu.kz done page

Your progress graph for this problem

Warmup-1 > front_back

prev | next | chance

Given a string, return a new string where the first and last chars have been exchanged.

front_back('code') → 'eodc'
front_back('a') → 'a'
front_back('ab') → 'ba'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def front_back(str):
  if len(str) <= 1:
    return str

  mid = str[1:len(str)-1]

  return str[len(str)-1] + mid + str[0]
```

Expected	Run		
front_back('code') → 'eodc'	'eodc'	OK	
front_back('a') → 'a'	'a'	OK	
front_back('ab') → 'ba'	'ba'	OK	
front_back('abc') → 'cba'	'cba'	OK	
front_back('') → ''	''	OK	
front_back('Chocolate') → 'ehocolatC'	'ehocolatC'	OK	
front_back('aavJ') → 'Java'	'Java'	OK	
front_back('hello') → 'oellh'	'oellh'	OK	

All Correct

next | chance

Python > Warmup-1

l_maratkyzy@kbtu.kz done page

Your progress graph for this problem

Go

Warmup-1 > pos_neg

prev | next | chance

Given 2 int values, return True if one is negative and one is positive. Except if the parameter "negative" is True, then return True only if both are negative.

pos_neg(1, -1, False) → True
pos_neg(-1, 1, False) → True
pos_neg(-4, -5, True) → True

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def pos_neg(a, b, negative):
  if negative:
    return (a < 0 and b < 0)
  else:
    return ((a < 0 and b > 0) or (a > 0 and b < 0))
```

Go

Editor font size %: 100

Expected	Run		
pos_neg(1, -1, False) → True	True	OK	
pos_neg(-1, 1, False) → True	True	OK	
pos_neg(-4, -5, True) → True	True	OK	
pos_neg(-4, -5, False) → False	False	OK	
pos_neg(-4, 5, False) → True	True	OK	
pos_neg(-4, 5, True) → False	False	OK	
pos_neg(1, 1, False) → False	False	OK	
pos_neg(-1, -1, False) → False	False	OK	
pos_neg(1, -1, True) → False	False	OK	
pos_neg(-1, 1, True) → False	False	OK	
pos_neg(1, 1, True) → False	False	OK	
pos_neg(-1, -1, True) → True	True	OK	
pos_neg(5, -5, False) → True	True	OK	
pos_neg(-6, 6, False) → True	True	OK	
pos_neg(-5, -6, False) → False	False	OK	
pos_neg(-2, -1, False) → False	False	OK	
pos_neg(1, 2, False) → False	False	OK	
pos_neg(-5, 6, True) → False	False	OK	
pos_neg(-5, -5, True) → True	True	OK	

Warmup-1 > parrot_trouble

prev | next | chance

We have a loud talking parrot. The "hour" parameter is the current hour time in the range 0..23. We are in trouble if the parrot is talking and the hour is before 7 or after 20. Return True if we are in trouble.

parrot_trouble(True, 6) → True
parrot_trouble(True, 7) → False
parrot_trouble(False, 6) → False

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def parrot_trouble(talking, hour):
  if talking and (7>hour or hour > 20):
    return True
  else:
    return False
```

Expected	Run		
parrot_trouble(True, 6) → True	True	OK	
parrot_trouble(True, 7) → False	False	OK	
parrot_trouble(False, 6) → False	False	OK	
parrot_trouble(True, 21) → True	True	OK	
parrot_trouble(False, 21) → False	False	OK	
parrot_trouble(False, 20) → False	False	OK	
parrot_trouble(True, 23) → True	True	OK	
parrot_trouble(False, 23) → False	False	OK	
parrot_trouble(True, 20) → False	False	OK	
parrot_trouble(False, 12) → False	False	OK	

All Correct

next | chance

Python > Warmup-1

l_maratkyzy@kbtu.kz done page

Your progress graph for this problem

Warmup-1 > monkey_trouble

prev | next | chance

We have two monkeys, a and b, and the parameters a_smile and b_smile indicate if each is smiling. We are in trouble if they are both smiling or if neither of them is smiling. Return True if we are in trouble.

monkey_trouble(True, True) → True
monkey_trouble(False, False) → True
monkey_trouble(True, False) → False

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def monkey_trouble(a_smile, b_smile):
  if a_smile and b_smile:
    return True
  if not a_smile and not b_smile:
    return True
  return False
```

Go

Expected	Run		
monkey_trouble(True, True) → True	True	OK	
monkey_trouble(False, False) → True	True	OK	
monkey_trouble(True, False) → False	False	OK	
monkey_trouble(False, True) → False	False	OK	

All Correct

next | chance

Python > Warmup-1

l_maratkyzy@kbtu.kz done page

Your progress graph for this problem

Warmup-1 > missing_char

prev | next | chance

Given a non-empty string and an int n, return a new string where the char at index n has been removed. The value of n will be a valid index of a char in the original string (i.e. n will be in the range 0..len(str)-1 inclusive).

missing_char('kitten', 1) → 'ktten'
missing_char('kitten', 0) → 'itten'
missing_char('kitten', 4) → 'kittn'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def missing_char(str, n):
  new_s =" "
  if n < len(str):
    for i in range(len(str)):
      if n == i:
        new_s = str.replace(str[i], "")
  return new_s
```

Expected	Run		
missing_char('kitten', 1) → 'ktten'	'ktten'	OK	
missing_char('kitten', 0) → 'itten'	'itten'	OK	
missing_char('kitten', 4) → 'kittn'	'kittn'	OK	
missing_char('Hi', 0) → 'i'	'i'	OK	
missing_char('Hi', 1) → 'H'	'H'	OK	
missing_char('code', 0) → 'ode'	'ode'	OK	
missing_char('code', 1) → 'cde'	'cde'	OK	
missing_char('code', 2) → 'coe'	'coe'	OK	
missing_char('code', 3) → 'cod'	'cod'	OK	
missing_char('chocolate', 8) → 'chocolat'	'chocolat'	OK	

All Correct

next | chance

Python > Warmup-1

l_maratkyzy@kbtu.kz done page

Your progress graph for this problem

Warmup-1 > near_hundred

prev | next | chance

Given an int n, return True if it is within 10 of 100 or 200. Note: abs(num) computes the absolute value of a number.

near_hundred(93) → True
near_hundred(90) → True
near_hundred(89) → False

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def near_hundred(n):
  if abs(100-n)<=10 or abs(200-n)<=10:
    return True
  else:
    return False
```

Go

Editor font size %: 100

Shorter output ☐

Expected	Run		
near_hundred(93) → True	True	OK	
near_hundred(90) → True	True	OK	
near_hundred(89) → False	False	OK	
near_hundred(110) → True	True	OK	
near_hundred(111) → False	False	OK	
near_hundred(121) → False	False	OK	
near_hundred(-101) → False	False	OK	
near_hundred(-209) → False	False	OK	
near_hundred(190) → True	True	OK	
near_hundred(209) → True	True	OK	
near_hundred(0) → False	False	OK	
near_hundred(5) → False	False	OK	
near_hundred(-50) → False	False	OK	
near_hundred(191) → True	True	OK	
near_hundred(189) → False	False	OK	
near_hundred(200) → True	True	OK	
near_hundred(210) → True	True	OK	
near_hundred(211) → False	False	OK	
near_hundred(290) → False	False	OK	

iv. WarmUp-2

Warmup-2 > string_match

prev | next | chance

Given 2 strings, a and b, return the number of the positions where they contain the same length 2 substring. So "xxcaazz" and "xxbaaz" yields 3, since the "xx", "aa", and "az" substrings appear in the same place in both strings.

string_match('xxcaazz', 'xxbaaz') → 3
string_match('abc', 'abc') → 2
string_match('abc', 'axc') → 0

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def string_match(a, b):

  shorter = min(len(a), len(b))
  count = 0

  for i in range(shorter-1):

    a_sub = a[i:i+2]
    b_sub = b[i:i+2]

    if a_sub == b_sub:
      count = count + 1

  return count
```

Expected	Run	
string_match('xxcaazz', 'xxbaaz') → 3	3	OK
string_match('abc', 'abc') → 2	2	OK
string_match('abc', 'axc') → 0	0	OK
string_match('hello', 'he') → 1	1	OK
string_match('he', 'hello') → 1	1	OK
string_match('h', 'hello') → 0	0	OK
string_match('', 'hello') → 0	0	OK
string_match('aabbccdd', 'abbbxxd') → 1	1	OK
string_match('aaxxaaxx', 'iaxxai') → 3	3	OK
string_match('iaxxai', 'aaxxaaxx') → 3	3	OK

All Correct

Warmup-2 > array123

prev | next | chance

Given an array of ints, return True if the sequence of numbers 1, 2, 3 appears in the array somewhere.

array123([1, 1, 2, 3, 1]) → True
array123([1, 1, 2, 4, 1]) → False
array123([1, 1, 2, 1, 2, 3]) → True

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def array123(nums):
  for i in range(len(nums)-2):

    if nums[i]==1 and nums[i+1]==2 and nums[i+2]==3:
      return True

  return False
```

Expected	Run	
array123([1, 1, 2, 3, 1]) → True	True	OK
array123([1, 1, 2, 4, 1]) → False	False	OK
array123([1, 1, 2, 1, 2, 3]) → True	True	OK
array123([1, 1, 2, 1, 2, 1]) → False	False	OK
array123([1, 2, 3, 1, 2, 3]) → True	True	OK
array123([1, 2, 3]) → True	True	OK
array123([1, 1, 1]) → False	False	OK
array123([1, 2]) → False	False	OK
array123([1]) → False	False	OK
array123([]) → False	False	OK

All Correct

Warmup-2 > array_front9

prev | next | chance

Given an array of ints, return True if one of the first 4 elements in the array is a 9. The array length may be less than 4.

array_front9([1, 2, 9, 3, 4]) → True
array_front9([1, 2, 3, 4, 9]) → False
array_front9([1, 2, 3, 4, 5]) → False

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def array_front9(nums):
  end = len(nums)

  if end > 4:
    end = 4

  for i in range(end):

    if nums[i] == 9:
      return True
  return False
```

Expected	Run		
array_front9([1, 2, 9, 3, 4]) → True	True	OK	
array_front9([1, 2, 3, 4, 9]) → False	False	OK	
array_front9([1, 2, 3, 4, 5]) → False	False	OK	
array_front9([9, 2, 3]) → True	True	OK	
array_front9([1, 9, 9]) → True	True	OK	
array_front9([1, 2, 3]) → False	False	OK	
array_front9([1, 9]) → True	True	OK	
array_front9([5, 5]) → False	False	OK	
array_front9([2]) → False	False	OK	
array_front9([9]) → True	True	OK	
array_front9([]) → False	False	OK	
array_front9([3, 9, 2, 3, 3]) → True	True	OK	

All Correct

next | chance

Warmup-2 > array_count9

prev | next | chance

Given an array of ints, return the number of 9's in the array.

array_count9([1, 2, 9]) → 1
array_count9([1, 9, 9]) → 2
array_count9([1, 9, 9, 3, 9]) → 3

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def array_count9(nums):
  count = 0

  for num in nums:
    if num == 9:
      count = count + 1

  return count
```

Expected	Run		
array_count9([1, 2, 9]) → 1	1	OK	
array_count9([1, 9, 9]) → 2	2	OK	
array_count9([1, 9, 9, 3, 9]) → 3	3	OK	
array_count9([1, 2, 3]) → 0	0	OK	
array_count9([]) → 0	0	OK	
array_count9([4, 2, 4, 3, 1]) → 0	0	OK	
array_count9([9, 2, 4, 3, 1]) → 1	1	OK	

All Correct

next | chance

Warmup-2 > last2

prev | next | chance

Given a string, return the count of the number of times that a substring length 2 appears in the string and also as the last 2 chars of the string, so "hixxxhi" yields 1 (we won't count the end substring).

last2('hixxhi') → 1
last2('xaxxaxaxx') → 1
last2('axxxaaxx') → 2

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def last2(str):
  if len(str) < 2:
    return 0

  last2 = str[len(str)-2:]

  count = 0

  for i in range(len(str)-2):
    sub = str[i:i+2]

    if sub == last2:
      count = count + 1

  return count
```

Expected	Run	
last2('hixxhi') → 1	1	OK
last2('xaxxaxaxx') → 1	1	OK
last2('axxxaaxx') → 2	2	OK
last2('xxaxxaxxaxx') → 3	3	OK
last2('xaxaxaxx') → 0	0	OK
last2('xxxx') → 2	2	OK
last2('13121312') → 1	1	OK
last2('11212') → 1	1	OK
last2('13121311') → 0	0	OK
last2('1717171') → 2	2	OK
last2('hi') → 0	0	OK
last2('h') → 0	0	OK
last2('') → 0	0	OK

All Correct

Warmup-2 > string_splosion

prev | next | chance

Given a non-empty string like "Code" return a string like "CCoCodCode".

string_splosion('Code') → 'CCoCodCode'
string_splosion('abc') → 'aababc'
string_splosion('ab') → 'aab'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def string_splosion(str):
  result = ""

  for i in range(len(str)):
    result = result + str[:i+1]
  return result
```

Expected	Run	
string_splosion('Code') → 'CCoCodCode'	'CCoCodCode'	OK
string_splosion('abc') → 'aababc'	'aababc'	OK
string_splosion('ab') → 'aab'	'aab'	OK
string_splosion('x') → 'x'	'x'	OK
string_splosion('fade') → 'ffafadfade'	'ffafadfade'	OK
string_splosion('There') → 'TThTheTherThere'	'TThTheTherThere'	OK
string_splosion('Kitten') → 'KKiKitKittKitteKitten'	'KKiKitKittKitteKitten'	OK
string_splosion('Bye') → 'BByBye'	'BByBye'	OK
string_splosion('Good') → 'GGoGooGood'	'GGoGooGood'	OK
string_splosion('Bad') → 'BBaBad'	'BBaBad'	OK

All Correct

Warmup-2 > string_bits

prev | next | chance

Given a string, return a new string made of every other char starting with the first, so "Hello" yields "Hlo".

string_bits('Hello') → 'Hlo'
string_bits('Hi') → 'H'
string_bits('Heeololeo') → 'Hello'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def string_bits(str):
  result = ""

  for i in range(len(str)):
    if i % 2 == 0:
      result = result + str[i]
  return result
```

Expected	Run		
string_bits('Hello') → 'Hlo'	'Hlo'	OK	
string_bits('Hi') → 'H'	'H'	OK	
string_bits('Heeololeo') → 'Hello'	'Hello'	OK	
string_bits('HiHiHi') → 'HHH'	'HHH'	OK	
string_bits('') → ''	''	OK	
string_bits('Greetings') → 'Getns'	'Getns'	OK	
string_bits('Chocoate') → 'Coot'	'Coot'	OK	
string_bits('pi') → 'p'	'p'	OK	
string_bits('Hello Kitten') → 'HloKte'	'HloKte'	OK	
string_bits('hxaxpxpxy') → 'happy'	'happy'	OK	

All Correct

next | chance

Warmup-2 > front_times

prev | next | chance

Given a string and a non-negative int n, we'll say that the front of the string is the first 3 chars, or whatever is there if the string is less than length 3. Return n copies of the front;

front_times('Chocolate', 2) → 'ChoCho'
front_times('Chocolate', 3) → 'ChoChoCho'
front_times('Abc', 3) → 'AbcAbcAbc'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def front_times(str, n):
  front_len = 3

  if front_len > len(str):
    front_len = len(str)

  front = str[:front_len]

  result = ""

  for i in range(n):
    result = result + front

  return result
```

Expected	Run		
front_times('Chocolate', 2) → 'ChoCho'	'ChoCho'	OK	
front_times('Chocolate', 3) → 'ChoChoCho'	'ChoChoCho'	OK	
front_times('Abc', 3) → 'AbcAbcAbc'	'AbcAbcAbc'	OK	
front_times('Ab', 4) → 'AbAbAbAb'	'AbAbAbAb'	OK	
front_times('A', 4) → 'AAAA'	'AAAA'	OK	
front_times('', 4) → ''	''	OK	
front_times('Abc', 0) → ''	''	OK	

All Correct

next | chance

Warmup-2 > string_times

prev | next | chance

Given a string and a non-negative int n, return a larger string that is n copies of the original string.

string_times('Hi', 2) → 'HiHi'
string_times('Hi', 3) → 'HiHiHi'
string_times('Hi', 1) → 'Hi'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def string_times(str, n):
  result = ""
  for i in range(n):
    result = result + str
  return result
```

Expected	Run		
string_times('Hi', 2) → 'HiHi'	'HiHi'	OK	
string_times('Hi', 3) → 'HiHiHi'	'HiHiHi'	OK	
string_times('Hi', 1) → 'Hi'	'Hi'	OK	
string_times('Hi', 0) → ''	''	OK	
string_times('Hi', 5) → 'HiHiHiHiHi'	'HiHiHiHiHi'	OK	
string_times('Oh Boy!', 2) → 'Oh Boy!Oh Boy!'	'Oh Boy!Oh Boy!'	OK	
string_times('x', 4) → 'xxxx'	'xxxx'	OK	
string_times('', 4) → ''	''	OK	
string_times('code', 2) → 'codecode'	'codecode'	OK	
string_times('code', 3) → 'codecodecode'	'codecodecode'	OK	

All Correct

V. String-1

String-1 > left2

prev | next | chance

Given a string, return a "rotated left 2" version where the first 2 chars are moved to the end. The string length will be at least 2.

left2('Hello') → 'lloHe'
left2('java') → 'vaja'
left2('Hi') → 'Hi'

Go ...Save, Compile, Run (ctrl-enter)

```
def left2(str):
  return str[2:] + str[:2]
```

Expected	Run		
left2('Hello') → 'lloHe'	'lloHe'	OK	
left2('java') → 'vaja'	'vaja'	OK	
left2('Hi') → 'Hi'	'Hi'	OK	
left2('code') → 'deco'	'deco'	OK	
left2('cat') → 'tca'	'tca'	OK	
left2('12345') → '34512'	'34512'	OK	
left2('Chocolate') → 'ocolateCh'	'ocolateCh'	OK	
left2('bricks') → 'icksbr'	'icksbr'	OK	
other tests		OK	

All Correct

String-1 > non_start

prev | next | chance

Given 2 strings, return their concatenation, except omit the first char of each. The strings will be at least length 1.

non_start('Hello', 'There') → 'ellohere'
non_start('java', 'code') → 'avaode'
non_start('shotl', 'java') → 'hotlava'

Go ...Save, Compile, Run (ctrl-enter)

```
def non_start(a, b):
    return a[1:] + b[1:]
```

Expected	Run		
non_start('Hello', 'There') → 'ellohere'	'ellohere'	OK	
non_start('java', 'code') → 'avaode'	'avaode'	OK	
non_start('shotl', 'java') → 'hotlava'	'hotlava'	OK	
non_start('ab', 'xy') → 'by'	'by'	OK	
non_start('ab', 'x') → 'b'	'b'	OK	
non_start('x', 'ac') → 'c'	'c'	OK	
non_start('a', 'x') → ''	''	OK	
non_start('kit', 'kat') → 'itat'	'itat'	OK	
non_start('mart', 'dart') → 'artart'	'artart'	OK	
other tests		OK	

All Correct

String-1 > combo_string

prev | next | chance

Given 2 strings, a and b, return a string of the form short+long+short, with the shorter string on the outside and the longer string on the inside. The strings will not be the same length, but they may be empty (length 0).

combo_string('Hello', 'hi') → 'hiHellohi'
combo_string('hi', 'Hello') → 'hiHellohi'
combo_string('aaa', 'b') → 'baaab'

Go ...Save, Compile, Run (ctrl-enter)

```
def combo_string(a, b):
    if len(a) < len(b):
        return a + b + a
    else:
        return b + a + b
```

Expected	Run		
combo_string('Hello', 'hi') → 'hiHellohi'	'hiHellohi'	OK	
combo_string('hi', 'Hello') → 'hiHellohi'	'hiHellohi'	OK	
combo_string('aaa', 'b') → 'baaab'	'baaab'	OK	
combo_string('b', 'aaa') → 'baaab'	'baaab'	OK	
combo_string('aaa', '') → 'aaa'	'aaa'	OK	
combo_string('', 'bb') → 'bb'	'bb'	OK	
combo_string('aaa', '1234') → 'aaa1234aaa'	'aaa1234aaa'	OK	
combo_string('aaa', 'bb') → 'bbaaabb'	'bbaaabb'	OK	
combo_string('a', 'bb') → 'abba'	'abba'	OK	
combo_string('bb', 'a') → 'abba'	'abba'	OK	
combo_string('xyz', 'ab') → 'abxyzab'	'abxyzab'	OK	
other tests		OK	

All Correct

String-1 > without_end

prev | next | chance

Given a string, return a version without the first and last char, so "Hello" yields "ell". The string length will be at least 2.

without_end('Hello') → 'ell'
without_end('java') → 'av'
without_end('coding') → 'odin'

Go ...Save, Compile, Run (ctrl-enter)

```
def without_end(str):
    return str[1:-1]
```

Expected	Run		
without_end('Hello') → 'ell'	'ell'	OK	
without_end('java') → 'av'	'av'	OK	
without_end('coding') → 'odin'	'odin'	OK	
without_end('code') → 'od'	'od'	OK	
without_end('ab') → ''	''	OK	
without_end('Chocolate!') → 'hocolate'	'hocolate'	OK	
without_end('kitten') → 'itte'	'itte'	OK	
without_end('woohoo') → 'ooho'	'ooho'	OK	
other tests		OK	

All Correct

String-1 > first_half

prev | next | chance

Given a string of even length, return the first half. So the string "WooHoo" yields "Woo".

first_half('WooHoo') → 'Woo'
first_half('HelloThere') → 'Hello'
first_half('abcdef') → 'abc'

Go ...Save, Compile, Run (ctrl-enter)

```
def first_half(str):
    half = len(str) // 2
    return str[:half]
```

Expected	Run		
first_half('WooHoo') → 'Woo'	'Woo'	OK	
first_half('HelloThere') → 'Hello'	'Hello'	OK	
first_half('abcdef') → 'abc'	'abc'	OK	
first_half('ab') → 'a'	'a'	OK	
first_half('') → ''	''	OK	
first_half('0123456789') → '01234'	'01234'	OK	
first_half('kitten') → 'kit'	'kit'	OK	
other tests		OK	

All Correct

String-1 > first_two

prev | next | chance

Given a string, return the string made of its first two chars, so the String "Hello" yields "He". If the string is shorter than length 2, return whatever there is, so "X" yields "X", and the empty string "" yields the empty string "".

first_two('Hello') → 'He'
first_two('abcdefg') → 'ab'
first_two('ab') → 'ab'

Go ...Save, Compile, Run (ctrl-enter)

```
def first_two(str):
  if len(str) >= 2:
    return str[:2]
  else:
    return str
```

Expected	Run		
first_two('Hello') → 'He'	'He'	OK	
first_two('abcdefg') → 'ab'	'ab'	OK	
first_two('ab') → 'ab'	'ab'	OK	
first_two('a') → 'a'	'a'	OK	
first_two('') → ''	''	OK	
first_two('Kitten') → 'Ki'	'Ki'	OK	
first_two('hi') → 'hi'	'hi'	OK	
first_two('hiya') → 'hi'	'hi'	OK	
other tests		OK	

All Correct

String-1 > extra_end

prev | next | chance

Given a string, return a new string made of 3 copies of the last 2 chars of the original string. The string length will be at least 2.

extra_end('Hello') → 'lololo'
extra_end('ab') → 'ababab'
extra_end('Hi') → 'HiHiHi'

Go ...Save, Compile, Run (ctrl-enter)

```
def extra_end(str):
    return str[-2:] * 3
```

Expected	Run		
extra_end('Hello') → 'lololo'	'lololo'	OK	
extra_end('ab') → 'ababab'	'ababab'	OK	
extra_end('Hi') → 'HiHiHi'	'HiHiHi'	OK	
extra_end('Candy') → 'dydydy'	'dydydy'	OK	
extra_end('Code') → 'dedede'	'dedede'	OK	
other tests		OK	

All Correct

String-1 > make_out_word

prev | next | chance

Given an "out" string length 4, such as "<<>>", and a word, return a new string where the word is in the middle of the out string, e.g. "<<word>>".

make_out_word('<<>>', 'Yay') → '<<Yay>>'
make_out_word('<<>>', 'WooHoo') → '<<WooHoo>>'
make_out_word('[[]]', 'word') → '[[word]]'

Go ...Save, Compile, Run (ctrl-enter)

```
def make_out_word(out, word):
    return out[0:2] + word + out[2:4]
```

Expected	Run		
make_out_word('<<>>', 'Yay') → '<<Yay>>'	'<<Yay>>'	OK	
make_out_word('<<>>', 'WooHoo') → '<<WooHoo>>'	'<<WooHoo>>'	OK	
make_out_word('[[]]', 'word') → '[[word]]'	'[[word]]'	OK	
make_out_word('HHoo', 'Hello') → 'HHHellooo'	'HHHellooo'	OK	
make_out_word('abyz', 'YAY') → 'abYAYyz'	'abYAYyz'	OK	
other tests		OK	

All Correct

String-1 > make_tags

prev | next | chance

The web is built with HTML strings like "Yay" which draws Yay as italic text. In this example, the "i" tag makes and which surround the word "Yay". Given tag and word strings, create the HTML string with tags around the word, e.g. "Yay".

make_tags('i', 'Yay') → 'Yay'
make_tags('i', 'Hello') → 'Hello'
make_tags('cite', 'Yay') → '<cite>Yay</cite>'

Go ...Save, Compile, Run (ctrl-enter)

```
def make_tags(tag, word):
    return "<" + tag + ">" + word + "</" + tag + ">"
```

Expected	Run	
make_tags('i', 'Yay') → 'Yay'	'Yay'	OK
make_tags('i', 'Hello') → 'Hello'	'Hello'	OK
make_tags('cite', 'Yay') → '<cite>Yay</cite>'	'<cite>Yay</cite>'	OK
make_tags('address', 'here') → '<address>here</address>'	'<address>here</address>'	OK
make_tags('body', 'Heart') → '<body>Heart</body>'	'<body>Heart</body>'	OK
make_tags('i', 'i') → 'i'	'i'	OK
make_tags('i', '') → ''	''	OK
other tests		OK

All Correct

next | chance

String-1 > make_abba

prev | next | chance

Given two strings, a and b, return the result of putting them together in the order abba, e.g. "Hi" and "Bye" returns "HiByeByeHi".

make_abba('Hi', 'Bye') → 'HiByeByeHi'
make_abba('Yo', 'Alice') → 'YoAliceAliceYo'
make_abba('What', 'Up') → 'WhatUpUpWhat'

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def make_abba(a, b):
    return a + b + b + a
```

Expected	Run	
make_abba('Hi', 'Bye') → 'HiByeByeHi'	'HiByeByeHi'	OK
make_abba('Yo', 'Alice') → 'YoAliceAliceYo'	'YoAliceAliceYo'	OK
make_abba('What', 'Up') → 'WhatUpUpWhat'	'WhatUpUpWhat'	OK
make_abba('aaa', 'bbb') → 'aaabbbbbbaaa'	'aaabbbbbbaaa'	OK
make_abba('x', 'y') → 'xyyx'	'xyyx'	OK
make_abba('x', '') → 'xx'	'xx'	OK
make_abba('', 'y') → 'yy'	'yy'	OK
make_abba('Bo', 'Ya') → 'BoYaYaBo'	'BoYaYaBo'	OK
make_abba('Ya', 'Ya') → 'YaYaYaYa'	'YaYaYaYa'	OK
other tests		OK

All Correct

String-1 > hello_name

prev | next | chance

Given a string name, e.g. "Bob", return a greeting of the form "Hello Bob!".

hello_name('Bob') → 'Hello Bob!'
hello_name('Alice') → 'Hello Alice!'
hello_name('X') → 'Hello X!'

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def hello_name(name):
    return "Hello "+name+"!"
```

Expected	Run	
hello_name('Bob') → 'Hello Bob!'	'Hello Bob!'	OK
hello_name('Alice') → 'Hello Alice!'	'Hello Alice!'	OK
hello_name('X') → 'Hello X!'	'Hello X!'	OK
hello_name('Dolly') → 'Hello Dolly!'	'Hello Dolly!'	OK
hello_name('Alpha') → 'Hello Alpha!'	'Hello Alpha!'	OK
hello_name('Omega') → 'Hello Omega!'	'Hello Omega!'	OK
hello_name('Goodbye') → 'Hello Goodbye!'	'Hello Goodbye!'	OK
hello_name('ho ho ho') → 'Hello ho ho ho!'	'Hello ho ho ho!'	OK
hello_name('xyz!') → 'Hello xyz!!'	'Hello xyz!!'	OK
hello_name('Hello') → 'Hello Hello!'	'Hello Hello!'	OK
other tests		OK

All Correct

List-1

List-1 > has23

prev | next | chance

Given an int array length 2, return True if it contains a 2 or a 3.

has23([2, 5]) → True
has23([4, 3]) → True
has23([4, 5]) → False

Go ...Save, Compile, Run (ctrl-enter)

```
def has23(nums):
    return 2 in nums or 3 in nums
```

Expected	Run		
has23([2, 5]) → True	True	OK	
has23([4, 3]) → True	True	OK	
has23([4, 5]) → False	False	OK	
has23([2, 2]) → True	True	OK	
has23([3, 2]) → True	True	OK	
has23([3, 3]) → True	True	OK	
has23([7, 7]) → False	False	OK	
has23([3, 9]) → True	True	OK	
has23([9, 5]) → False	False	OK	
other tests		OK	

All Correct

List-1 > make_ends

prev | next | chance

Given an array of ints, return a new array length 2 containing the first and last elements from the original array. The original array will be length 1 or more.

make_ends([1, 2, 3]) → [1, 3]
make_ends([1, 2, 3, 4]) → [1, 4]
make_ends([7, 4, 6, 2]) → [7, 2]

Go ...Save, Compile, Run (ctrl-enter)

```
def make_ends(nums):
    return [nums[0], nums[-1]]
```

Expected	Run		
make_ends([1, 2, 3]) → [1, 3]	[1, 3]	OK	
make_ends([1, 2, 3, 4]) → [1, 4]	[1, 4]	OK	
make_ends([7, 4, 6, 2]) → [7, 2]	[7, 2]	OK	
make_ends([1, 2, 2, 2, 2, 2, 2, 3]) → [1, 3]	[1, 3]	OK	
make_ends([7, 4]) → [7, 4]	[7, 4]	OK	
make_ends([7]) → [7, 7]	[7, 7]	OK	
make_ends([5, 2, 9]) → [5, 9]	[5, 9]	OK	
make_ends([2, 3, 4, 1]) → [2, 1]	[2, 1]	OK	
other tests		OK	

All Correct

List-1 > middle_way

prev | next | chance

Given 2 int arrays, a and b, each length 3, return a new array length 2 containing their middle elements.

middle_way([1, 2, 3], [4, 5, 6]) → [2, 5]
middle_way([7, 7, 7], [3, 8, 0]) → [7, 8]
middle_way([5, 2, 9], [1, 4, 5]) → [2, 4]

Go ...Save, Compile, Run (ctrl-enter)

```
def middle_way(a, b):
    return [a[1], b[1]]
```

Expected	Run		
middle_way([1, 2, 3], [4, 5, 6]) → [2, 5]	[2, 5]	OK	
middle_way([7, 7, 7], [3, 8, 0]) → [7, 8]	[7, 8]	OK	
middle_way([5, 2, 9], [1, 4, 5]) → [2, 4]	[2, 4]	OK	
middle_way([1, 9, 7], [4, 8, 8]) → [9, 8]	[9, 8]	OK	
middle_way([1, 2, 3], [3, 1, 4]) → [2, 1]	[2, 1]	OK	
middle_way([1, 2, 3], [4, 1, 1]) → [2, 1]	[2, 1]	OK	
other tests		OK	

All Correct

List-1 > sum2

prev | next | chance

Given an array of ints, return the sum of the first 2 elements in the array. If the array length is less than 2, just sum up the elements that exist, returning 0 if the array is length 0.

sum2([1, 2, 3]) → 3
sum2([1, 1]) → 2
sum2([1, 1, 1, 1]) → 2

Go ...Save, Compile, Run (ctrl-enter)

```
def sum2(nums):
    return sum(nums[:2])
```

Expected	Run		
sum2([1, 2, 3]) → 3	3	OK	
sum2([1, 1]) → 2	2	OK	
sum2([1, 1, 1, 1]) → 2	2	OK	
sum2([1, 2]) → 3	3	OK	
sum2([1]) → 1	1	OK	
sum2([]) → 0	0	OK	
sum2([4, 5, 6]) → 9	9	OK	
sum2([4]) → 4	4	OK	
other tests		OK	

All Correct

List-1 > max_end3

prev | next | chance

Given an array of ints length 3, figure out which is larger, the first or last element in the array, and set all the other elements to be that value. Return the changed array.

max_end3([1, 2, 3]) → [3, 3, 3]
max_end3([11, 5, 9]) → [11, 11, 11]
max_end3([2, 11, 3]) → [3, 3, 3]

Go ...Save, Compile, Run (ctrl-enter)

```
def max_end3(nums):
  large = max(nums[0], nums[2])
  nums[0] = large
  nums[1] = large
  nums[2] = large
  return nums
```

Expected	Run		
max_end3([1, 2, 3]) → [3, 3, 3]	[3, 3, 3]	OK	
max_end3([11, 5, 9]) → [11, 11, 11]	[11, 11, 11]	OK	
max_end3([2, 11, 3]) → [3, 3, 3]	[3, 3, 3]	OK	
max_end3([11, 3, 3]) → [11, 11, 11]	[11, 11, 11]	OK	
max_end3([3, 11, 11]) → [11, 11, 11]	[11, 11, 11]	OK	
max_end3([2, 2, 2]) → [2, 2, 2]	[2, 2, 2]	OK	
max_end3([2, 11, 2]) → [2, 2, 2]	[2, 2, 2]	OK	
max_end3([0, 0, 1]) → [1, 1, 1]	[1, 1, 1]	OK	
other tests		OK	

All Correct

List-1 > reverse3

prev | next | chance

Given an array of ints length 3, return a new array with the elements in reverse order, so {1, 2, 3} becomes {3, 2, 1}.

reverse3([1, 2, 3]) → [3, 2, 1]
reverse3([5, 11, 9]) → [9, 11, 5]
reverse3([7, 0, 0]) → [0, 0, 7]

Go ...Save, Compile, Run (ctrl-enter)

```
def reverse3(nums):
    return nums[::-1]
```

Expected	Run		
reverse3([1, 2, 3]) → [3, 2, 1]	[3, 2, 1]	OK	
reverse3([5, 11, 9]) → [9, 11, 5]	[9, 11, 5]	OK	
reverse3([7, 0, 0]) → [0, 0, 7]	[0, 0, 7]	OK	
reverse3([2, 1, 2]) → [2, 1, 2]	[2, 1, 2]	OK	
reverse3([1, 2, 1]) → [1, 2, 1]	[1, 2, 1]	OK	
reverse3([2, 11, 3]) → [3, 11, 2]	[3, 11, 2]	OK	
reverse3([0, 6, 5]) → [5, 6, 0]	[5, 6, 0]	OK	
reverse3([7, 2, 3]) → [3, 2, 7]	[3, 2, 7]	OK	
other tests		OK	

All Correct

List-1 > rotate_left3

prev | next | chance

Given an array of ints length 3, return an array with the elements "rotated left" so {1, 2, 3} yields {2, 3, 1}.

rotate_left3([1, 2, 3]) → [2, 3, 1]
rotate_left3([5, 11, 9]) → [11, 9, 5]
rotate_left3([7, 0, 0]) → [0, 0, 7]

Go ...Save, Compile, Run (ctrl-enter)

```
def rotate_left3(nums):
    return [nums[1], nums[2], nums[0]]
```

Expected	Run	
rotate_left3([1, 2, 3]) → [2, 3, 1]	[2, 3, 1]	OK
rotate_left3([5, 11, 9]) → [11, 9, 5]	[11, 9, 5]	OK
rotate_left3([7, 0, 0]) → [0, 0, 7]	[0, 0, 7]	OK
rotate_left3([1, 2, 1]) → [2, 1, 1]	[2, 1, 1]	OK
rotate_left3([0, 0, 1]) → [0, 1, 0]	[0, 1, 0]	OK
other tests		OK

All Correct

List-1 > sum3

prev | next | chance

Given an array of ints length 3, return the sum of all the elements.

sum3([1, 2, 3]) → 6
sum3([5, 11, 2]) → 18
sum3([7, 0, 0]) → 7

Go ...Save, Compile, Run (ctrl-enter)

```
def sum3(nums):
    return sum(nums)
```

Expected	Run	
sum3([1, 2, 3]) → 6	6	OK
sum3([5, 11, 2]) → 18	18	OK
sum3([7, 0, 0]) → 7	7	OK
sum3([1, 2, 1]) → 4	4	OK
sum3([1, 1, 1]) → 3	3	OK
sum3([2, 7, 2]) → 11	11	OK

All Correct

List-1 > common_end

prev | next | chance

Given 2 arrays of ints, a and b, return True if they have the same first element or they have the same last element. Both arrays will be length 1 or more.

common_end([1, 2, 3], [7, 3]) → True
common_end([1, 2, 3], [7, 3, 2]) → False
common_end([1, 2, 3], [1, 3]) → True

Go ...Save, Compile, Run (ctrl-enter)

```
def common_end(a, b):
    return a[0] == b[0] or a[-1] == b[-1]
```

Expected	Run	
common_end([1, 2, 3], [7, 3]) → True	True	OK
common_end([1, 2, 3], [7, 3, 2]) → False	False	OK
common_end([1, 2, 3], [1, 3]) → True	True	OK
common_end([1, 2, 3], [1]) → True	True	OK
common_end([1, 2, 3], [2]) → False	False	OK
other tests		OK

All Correct

next | chance

List-1 > make_pi

prev | next | chance

Return an int array length 3 containing the first 3 digits of pi, {3, 1, 4}.

make_pi() → [3, 1, 4]

Go ...Save, Compile, Run (ctrl-enter)

```
def make_pi():
    return [3, 1, 4]
```

Expected	Run		
make_pi() → [3, 1, 4]	[3, 1, 4]	OK	

All Correct

next | chance

List-1 > same_first_last

prev | next | chance

Given an array of ints, return True if the array is length 1 or more, and the first element and the last element are equal.

same_first_last([1, 2, 3]) → False
same_first_last([1, 2, 3, 1]) → True
same_first_last([1, 2, 1]) → True

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def same_first_last(nums):
    return len(nums) >= 1 and nums[0] == nums[-1]
```

Expected	Run		
same_first_last([1, 2, 3]) → False	False	OK	
same_first_last([1, 2, 3, 1]) → True	True	OK	
same_first_last([1, 2, 1]) → True	True	OK	
same_first_last([7]) → True	True	OK	
same_first_last([]) → False	False	OK	
same_first_last([1, 2, 3, 4, 5, 1]) → True	True	OK	
same_first_last([1, 2, 3, 4, 5, 13]) → False	False	OK	
same_first_last([13, 2, 3, 4, 5, 13]) → True	True	OK	
same_first_last([7, 7]) → True	True	OK	
other tests		OK	

All Correct

List-1 > first_last6

prev | next | chance

Given an array of ints, return True if 6 appears as either the first or last element in the array. The array will be length 1 or more.

first_last6([1, 2, 6]) → True
first_last6([6, 1, 2, 3]) → True
first_last6([13, 6, 1, 2, 3]) → False

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def first_last6(nums):
    return nums[0] == 6 or nums[-1] == 6
```

Expected	Run		
first_last6([1, 2, 6]) → True	True	OK	
first_last6([6, 1, 2, 3]) → True	True	OK	
first_last6([13, 6, 1, 2, 3]) → False	False	OK	
first_last6([13, 6, 1, 2, 6]) → True	True	OK	
first_last6([3, 2, 1]) → False	False	OK	
first_last6([3, 6, 1]) → False	False	OK	
first_last6([3, 6]) → True	True	OK	
first_last6([6]) → True	True	OK	
first_last6([3]) → False	False	OK	
first_last6([5, 6]) → True	True	OK	
first_last6([5, 5]) → False	False	OK	
first_last6([1, 2, 3, 4, 6]) → True	True	OK	
first_last6([1, 2, 3, 4]) → False	False	OK	
other tests		OK	

All Correct

Go

Logic-1

Logic-1 > near_ten

prev | next | chance

Given a non-negative number "num", return True if num is within 2 of a multiple of 10. Note: (a % b) is the remainder of dividing a by b, so (7 % 5) is 2. See also: Introduction to Mod

near_ten(12) → True
near_ten(17) → False
near_ten(19) → True

Go ...Save, Compile, Run (ctrl-enter)

```
def near_ten(num):
    remainder = num % 10
    return remainder <= 2 or remainder >= 8
```

Go

Expected	Run		
near_ten(12) → True	True	OK	
near_ten(17) → False	False	OK	
near_ten(19) → True	True	OK	
near_ten(31) → True	True	OK	
near_ten(6) → False	False	OK	
near_ten(10) → True	True	OK	
near_ten(11) → True	True	OK	
near_ten(21) → True	True	OK	
near_ten(22) → True	True	OK	
near_ten(23) → False	False	OK	
near_ten(54) → False	False	OK	
near_ten(155) → False	False	OK	
near_ten(158) → True	True	OK	
near_ten(3) → False	False	OK	
near_ten(1) → True	True	OK	
other tests		OK	

All Correct

Logic-1 > in1to10

prev | next | chance

Given a number n, return True if n is in the range 1..10, inclusive. Unless outside_mode is True, in which case return True if the number is less or equal to 1, or greater or equal to 10.

in1to10(5, False) → True
in1to10(11, False) → False
in1to10(11, True) → True

Go ...Save, Compile, Run (ctrl-enter)

```
def in1to10(n, outside_mode):
    if outside_mode:
        return n <= 1 or n >= 10
    return 1 <= n <= 10
```

Go

Expected	Run		
in1to10(5, False) → True	True	OK	
in1to10(11, False) → False	False	OK	
in1to10(11, True) → True	True	OK	
in1to10(10, False) → True	True	OK	
in1to10(10, True) → True	True	OK	
in1to10(9, False) → True	True	OK	
in1to10(9, True) → False	False	OK	
in1to10(1, False) → True	True	OK	
in1to10(1, True) → True	True	OK	
in1to10(0, False) → False	False	OK	
in1to10(0, True) → True	True	OK	
in1to10(-1, False) → False	False	OK	
in1to10(-1, True) → True	True	OK	
in1to10(99, False) → False	False	OK	
in1to10(-99, True) → True	True	OK	
other tests		OK	

All Correct

Logic-1 > love6

prev | next | chance

The number 6 is a truly great number. Given two int values, a and b, return True if either one is 6. Or if their sum or difference is 6. Note: the function abs(num) computes the absolute value of a number.

love6(6, 4) → True
love6(4, 5) → False
love6(1, 5) → True

Go ...Save, Compile, Run (ctrl-enter)

```
def love6(a, b):
    return a == 6 or b == 6 or a + b == 6 or abs(a - b) == 6
```

Go

Editor font size %: 100

Shorter output ☐

Expected	Run		
love6(6, 4) → True	True	OK	
love6(4, 5) → False	False	OK	
love6(1, 5) → True	True	OK	
love6(1, 6) → True	True	OK	
love6(1, 8) → False	False	OK	
love6(1, 7) → True	True	OK	
love6(7, 5) → False	False	OK	
love6(8, 2) → True	True	OK	
love6(6, 6) → True	True	OK	
love6(-6, 2) → False	False	OK	
love6(-4, -10) → True	True	OK	
love6(-7, 1) → False	False	OK	
love6(7, -1) → True	True	OK	
love6(-6, 12) → True	True	OK	
love6(-2, -4) → False	False	OK	
love6(7, 1) → True	True	OK	
love6(0, 9) → False	False	OK	
love6(8, 3) → False	False	OK	
love6(3, 3) → True	True	OK	
love6(3, 4) → False	False	OK	
other tests		OK	

All Correct

Logic-1 > alarm_clock

prev | next | chance

Given a day of the week encoded as 0=Sun, 1=Mon, 2=Tue, ...6=Sat, and a boolean indicating if we are on vacation, return a string of the form "7:00" indicating when the alarm clock should ring. Weekdays, the alarm should be "7:00" and on the weekend it should be "10:00". Unless we are on vacation -- then on weekdays it should be "10:00" and weekends it should be "off".

alarm_clock(1, False) → '7:00'
alarm_clock(5, False) → '7:00'
alarm_clock(0, False) → '10:00'

Go ...Save, Compile, Run (ctrl-enter)

```
def alarm_clock(day, vacation):
    is_weekend = (day == 0 or day == 6)
    if vacation:
        return "off" if is_weekend else "10:00"
    return "10:00" if is_weekend else "7:00"
```

Expected	Run		
alarm_clock(1, False) → '7:00'	'7:00'	OK	
alarm_clock(5, False) → '7:00'	'7:00'	OK	
alarm_clock(0, False) → '10:00'	'10:00'	OK	
alarm_clock(6, False) → '10:00'	'10:00'	OK	
alarm_clock(0, True) → 'off'	'off'	OK	
alarm_clock(6, True) → 'off'	'off'	OK	
alarm_clock(1, True) → '10:00'	'10:00'	OK	
alarm_clock(3, True) → '10:00'	'10:00'	OK	
alarm_clock(5, True) → '10:00'	'10:00'	OK	
other tests		OK	

All Correct

Logic-1 > sorta_sum

prev | next | chance

Given 2 ints, a and b, return their sum. However, sums in the range 10..19 inclusive, are forbidden, so in that case just return 20.

sorta_sum(3, 4) → 7
sorta_sum(9, 4) → 20
sorta_sum(10, 11) → 21

Go ...Save, Compile, Run (ctrl-enter)

```
def sorta_sum(a, b):
    total = a + b
    return 20 if 10 <= total <= 19 else total
```

Expected	Run		
sorta_sum(3, 4) → 7	7	OK	
sorta_sum(9, 4) → 20	20	OK	
sorta_sum(10, 11) → 21	21	OK	
sorta_sum(12, -3) → 9	9	OK	
sorta_sum(-3, 12) → 9	9	OK	
sorta_sum(4, 5) → 9	9	OK	
sorta_sum(4, 6) → 20	20	OK	
sorta_sum(14, 7) → 21	21	OK	
sorta_sum(14, 6) → 20	20	OK	
other tests		OK	

All Correct

Logic-1 > caught_speeding

prev | next | chance

You are driving a little too fast, and a police officer stops you. Write code to compute the result, encoded as an int value: 0=no ticket, 1=small ticket, 2=big ticket. If speed is 60 or less, the result is 0. If speed is between 61 and 80 inclusive, the result is 1. If speed is 81 or more, the result is 2. Unless it is your birthday -- on that day, your speed can be 5 higher in all cases.

caught_speeding(60, False) → 0
caught_speeding(65, False) → 1
caught_speeding(65, True) → 0

Go ...Save, Compile, Run (ctrl-enter)

```
def caught_speeding(speed, is_birthday):
    bonus = 5 if is_birthday else 0

    if speed <= 60 + bonus:
        return 0
    elif 61 + bonus <= speed <= 80 + bonus:
        return 1
    else:
        return 2
```

Expected	Run		
caught_speeding(60, False) → 0	0	OK	
caught_speeding(65, False) → 1	1	OK	
caught_speeding(65, True) → 0	0	OK	
caught_speeding(80, False) → 1	1	OK	
caught_speeding(85, False) → 2	2	OK	
caught_speeding(85, True) → 1	1	OK	
caught_speeding(70, False) → 1	1	OK	
caught_speeding(75, False) → 1	1	OK	
caught_speeding(75, True) → 1	1	OK	
caught_speeding(40, False) → 0	0	OK	
caught_speeding(40, True) → 0	0	OK	
caught_speeding(90, False) → 2	2	OK	
other tests		OK	

All Correct

Logic-1 > squirrel_play

prev | next | chance

The squirrels in Palo Alto spend most of the day playing. In particular, they play if the temperature is between 60 and 90 (inclusive). Unless it is summer, then the upper limit is 100 instead of 90. Given an int temperature and a boolean is_summer, return True if the squirrels play and False otherwise.

squirrel_play(70, False) → True
squirrel_play(95, False) → False
squirrel_play(95, True) → True

Go ...Save, Compile, Run (ctrl-enter)

```
def squirrel_play(temp, is_summer):
    upper_limit = 100 if is_summer else 90
    return 60 <= temp <= upper_limit
```

Expected	Run		
squirrel_play(70, False) → True	True	OK	
squirrel_play(95, False) → False	False	OK	
squirrel_play(95, True) → True	True	OK	
squirrel_play(90, False) → True	True	OK	
squirrel_play(90, True) → True	True	OK	
squirrel_play(50, False) → False	False	OK	
squirrel_play(50, True) → False	False	OK	
squirrel_play(100, False) → False	False	OK	
squirrel_play(100, True) → True	True	OK	
squirrel_play(105, True) → False	False	OK	
squirrel_play(59, False) → False	False	OK	
squirrel_play(59, True) → False	False	OK	
squirrel_play(60, False) → True	True	OK	
other tests		OK	

All Correct

Logic-1 > date_fashion

prev | next | chance

You and your date are trying to get a table at a restaurant. The parameter "you" is the stylishness of your clothes, in the range 0..10, and "date" is the stylishness of your date's clothes. The result getting the table is encoded as an int value with 0=no, 1=maybe, 2=yes. If either of you is very stylish, 8 or more, then the result is 2 (yes). With the exception that if either of you has style of 2 or less, then the result is 0 (no). Otherwise the result is 1 (maybe).

date_fashion(5, 10) → 2
date_fashion(5, 2) → 0
date_fashion(5, 5) → 1

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def date_fashion(you, date):
  if (you <= 2) or (date <= 2):
    return 0
  elif (you >= 8) or (date >= 8):
    return 2
  else:
    return 1
```

Expected	Run		
date_fashion(5, 10) → 2	2	OK	
date_fashion(5, 2) → 0	0	OK	
date_fashion(5, 5) → 1	1	OK	
date_fashion(3, 3) → 1	1	OK	
date_fashion(10, 2) → 0	0	OK	
date_fashion(2, 9) → 0	0	OK	
date_fashion(9, 9) → 2	2	OK	
date_fashion(10, 5) → 2	2	OK	
date_fashion(2, 2) → 0	0	OK	
date_fashion(3, 7) → 1	1	OK	
date_fashion(2, 7) → 0	0	OK	
date_fashion(6, 2) → 0	0	OK	
other tests		OK	

All Correct

Logic-1 > cigar_party

prev | next | chance

When squirrels get together for a party, they like to have cigars. A squirrel party is successful when the number of cigars is between 40 and 60, inclusive. Unless it is the weekend, in which case there is no upper bound on the number of cigars. Return True if the party with the given values is successful, or False otherwise.

cigar_party(30, False) → False
cigar_party(50, False) → True
cigar_party(70, True) → True

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def cigar_party(cigars, is_weekend):
  if is_weekend:
    return (cigars >= 40)
  else:
    return (cigars >= 40 and cigars <= 60)
```

Expected	Run		
cigar_party(30, False) → False	False	OK	
cigar_party(50, False) → True	True	OK	
cigar_party(70, True) → True	True	OK	
cigar_party(30, True) → False	False	OK	
cigar_party(50, True) → True	True	OK	
cigar_party(60, False) → True	True	OK	
cigar_party(61, False) → False	False	OK	
cigar_party(40, False) → True	True	OK	
cigar_party(39, False) → False	False	OK	
cigar_party(40, True) → True	True	OK	
cigar_party(39, True) → False	False	OK	
other tests		OK	

All Correct

Logic-2

Logic-2 > make_chocolate

prev | next | chance

We want make a package of **goal** kilos of chocolate. We have small bars (1 kilo each) and big bars (5 kilos each). Return the number of small bars to use, assuming we always use big bars before small bars. Return -1 if it can't be done.

make_chocolate(4, 1, 9) → 4
make_chocolate(4, 1, 10) → -1
make_chocolate(4, 1, 7) → 2

Go ...Save, Compile, Run (ctrl-enter)

```
def make_chocolate(small, big, goal):
    max_big = goal // 5
    if big >= max_big:
        goal -= max_big * 5
    else:
        goal -= big * 5

    if goal <= small:
        return goal
    return -1
```

Go

Expected	Run		
make_chocolate(4, 1, 9) → 4	4	OK	
make_chocolate(4, 1, 10) → -1	-1	OK	
make_chocolate(4, 1, 7) → 2	2	OK	
make_chocolate(6, 2, 7) → 2	2	OK	
make_chocolate(4, 1, 5) → 0	0	OK	
make_chocolate(4, 1, 4) → 4	4	OK	
make_chocolate(5, 4, 9) → 4	4	OK	
make_chocolate(9, 3, 18) → 3	3	OK	
make_chocolate(3, 1, 9) → -1	-1	OK	
make_chocolate(1, 2, 7) → -1	-1	OK	
make_chocolate(1, 2, 6) → 1	1	OK	
make_chocolate(1, 2, 5) → 0	0	OK	
make_chocolate(6, 1, 10) → 5	5	OK	
make_chocolate(6, 1, 11) → 6	6	OK	
make_chocolate(6, 1, 12) → -1	-1	OK	
make_chocolate(6, 1, 13) → -1	-1	OK	
make_chocolate(6, 2, 10) → 0	0	OK	
make_chocolate(6, 2, 11) → 1	1	OK	
make_chocolate(6, 2, 12) → 2	2	OK	

Logic-2 > close_far

prev | next | chance

Given three ints, a b c, return True if one of b or c is "close" (differing from a by at most 1), while the other is "far", differing from both other values by 2 or more. Note: abs(num) computes the absolute value of a number.

close_far(1, 2, 10) → True
close_far(1, 2, 3) → False
close_far(4, 1, 3) → True

Go ...Save, Compile, Run (ctrl-enter)

```
def close_far(a, b, c):
    def is_close(x, y):
        return abs(x - y) <= 1

    def is_far(x, y, z):
        return abs(x - y) >= 2 and abs(x - z) >= 2

    if is_close(a, b):
        return is_far(c, a, b)
    if is_close(a, c):
        return is_far(b, a, c)
    return False
```

Expected	Run		
close_far(1, 2, 10) → True	True	OK	
close_far(1, 2, 3) → False	False	OK	
close_far(4, 1, 3) → True	True	OK	
close_far(4, 5, 3) → False	False	OK	
close_far(4, 3, 5) → False	False	OK	
close_far(-1, 10, 0) → True	True	OK	
close_far(0, -1, 10) → True	True	OK	
close_far(10, 10, 8) → True	True	OK	
close_far(10, 8, 9) → False	False	OK	
close_far(8, 9, 10) → False	False	OK	
close_far(8, 9, 7) → False	False	OK	
close_far(8, 6, 9) → True	True	OK	
other tests		OK	

All Correct

Logic-2 > round_sum

prev | next | chance

For this problem, we'll round an int value up to the next multiple of 10 if its rightmost digit is 5 or more, so 15 rounds up to 20. Alternately, round down to the previous multiple of 10 if its rightmost digit is less than 5, so 12 rounds down to 10. Given 3 ints, a b c, return the sum of their rounded values. To avoid code repetition, write a separate helper "def round10(num):" and call it 3 times. Write the helper entirely below and at the same indent level as round_sum().

round_sum(16, 17, 18) → 60
round_sum(12, 13, 14) → 30
round_sum(6, 4, 4) → 10

Go ...Save, Compile, Run (ctrl-enter)

```
def round_sum(a, b, c):
  return round10(a) + round10(b) + round10(c)

def round10(num):
  mod = num % 10
  num -= mod
  if mod >= 5: num += 10
  return num
```

Expected	Run		
round_sum(16, 17, 18) → 60	60	OK	
round_sum(12, 13, 14) → 30	30	OK	
round_sum(6, 4, 4) → 10	10	OK	
round_sum(4, 6, 5) → 20	20	OK	
round_sum(4, 4, 6) → 10	10	OK	
round_sum(9, 4, 4) → 10	10	OK	
round_sum(0, 0, 1) → 0	0	OK	
round_sum(0, 9, 0) → 10	10	OK	
round_sum(10, 10, 19) → 40	40	OK	
round_sum(20, 30, 40) → 90	90	OK	
round_sum(45, 21, 30) → 100	100	OK	
round_sum(23, 11, 26) → 60	60	OK	
round_sum(23, 24, 25) → 70	70	OK	
round_sum(25, 24, 25) → 80	80	OK	
round_sum(23, 24, 29) → 70	70	OK	
round_sum(11, 24, 36) → 70	70	OK	
round_sum(24, 36, 32) → 90	90	OK	
round_sum(14, 12, 26) → 50	50	OK	
round_sum(12, 10, 24) → 40	40	OK	

Logic-2 > no_teen_sum

prev | next | chance

Given 3 int values, a b c, return their sum. However, if any of the values is a teen -- in the range 13..19 inclusive -- then that value counts as 0, except 15 and 16 do not count as a teens. Write a separate helper "def fix_teen(n):"that takes in an int value and returns that value fixed for the teen rule. In this way, you avoid repeating the teen code 3 times (i.e. "decomposition"). Define the helper below and at the same indent level as the main no_teen_sum().

no_teen_sum(1, 2, 3) → 6
no_teen_sum(2, 13, 1) → 3
no_teen_sum(2, 1, 14) → 3

Go ...Save, Compile, Run (ctrl-enter)

```
def no_teen_sum(a, b, c):
    return fix_teen(a) + fix_teen(b) + fix_teen(c)

def fix_teen(n):
    if 13 <= n <= 19 and n != 15 and n != 16:
        return 0
    return n
```

Expected	Run		
no_teen_sum(1, 2, 3) → 6	6	OK	
no_teen_sum(2, 13, 1) → 3	3	OK	
no_teen_sum(2, 1, 14) → 3	3	OK	
no_teen_sum(2, 1, 15) → 18	18	OK	
no_teen_sum(2, 1, 16) → 19	19	OK	
no_teen_sum(2, 1, 17) → 3	3	OK	
no_teen_sum(17, 1, 2) → 3	3	OK	
no_teen_sum(2, 15, 2) → 19	19	OK	
no_teen_sum(16, 17, 18) → 16	16	OK	
no_teen_sum(17, 18, 19) → 0	0	OK	
no_teen_sum(15, 16, 1) → 32	32	OK	
no_teen_sum(15, 15, 19) → 30	30	OK	
no_teen_sum(15, 19, 16) → 31	31	OK	
no_teen_sum(5, 17, 18) → 5	5	OK	
no_teen_sum(17, 18, 16) → 16	16	OK	
no_teen_sum(17, 19, 18) → 0	0	OK	
other tests		OK	

All Correct

Logic-2 > lucky_sum

prev | next | chance

Given 3 int values, a b c, return their sum. However, if one of the values is 13 then it does not count towards the sum and values to its right do not count. So for example, if b is 13, then both b and c do not count.

lucky_sum(1, 2, 3) → 6
lucky_sum(1, 2, 13) → 3
lucky_sum(1, 13, 3) → 1

Go ...Save, Compile, Run (ctrl-enter)

```
def lucky_sum(a, b, c):
    if a == 13: return 0
    if b == 13: return a
    if c == 13: return a + b
    return a + b + c
```

Expected	Run		
lucky_sum(1, 2, 3) → 6	6	OK	
lucky_sum(1, 2, 13) → 3	3	OK	
lucky_sum(1, 13, 3) → 1	1	OK	
lucky_sum(1, 13, 13) → 1	1	OK	
lucky_sum(6, 5, 2) → 13	13	OK	
lucky_sum(13, 2, 3) → 0	0	OK	
lucky_sum(13, 2, 13) → 0	0	OK	
lucky_sum(13, 13, 2) → 0	0	OK	
lucky_sum(9, 4, 13) → 13	13	OK	
lucky_sum(8, 13, 2) → 8	8	OK	
lucky_sum(7, 2, 1) → 10	10	OK	
lucky_sum(3, 3, 13) → 6	6	OK	
other tests		OK	

All Correct

Logic-2 > lone_sum

prev | next | chance

Given 3 int values, a b c, return their sum. However, if one of the values is the same as another of the values, it does not count towards the sum.

lone_sum(1, 2, 3) → 6
lone_sum(3, 2, 3) → 2
lone_sum(3, 3, 3) → 0

Go ...Save, Compile, Run (ctrl-enter)

```
def lone_sum(a, b, c):
  sum = 0
  if a != b and a != c: sum += a
  if b != a and b != c: sum += b
  if c != a and c != b: sum += c

  return sum
```

Expected	Run		
lone_sum(1, 2, 3) → 6	6	OK	
lone_sum(3, 2, 3) → 2	2	OK	
lone_sum(3, 3, 3) → 0	0	OK	
lone_sum(9, 2, 2) → 9	9	OK	
lone_sum(2, 2, 9) → 9	9	OK	
lone_sum(2, 9, 2) → 9	9	OK	
lone_sum(2, 9, 3) → 14	14	OK	
lone_sum(4, 2, 3) → 9	9	OK	
lone_sum(1, 3, 1) → 3	3	OK	
other tests		OK	

All Correct

Logic-2 > make_bricks

prev | next | chance

We want to make a row of bricks that is **goal** inches long. We have a number of small bricks (1 inch each) and big bricks (5 inches each). Return True if it is possible to make the goal by choosing from the given bricks. This is a little harder than it looks and can be done without any loops. See also: Introduction to MakeBricks

make_bricks(3, 1, 8) → True
make_bricks(3, 1, 9) → False
make_bricks(3, 2, 10) → True

Go ...Save, Compile, Run (ctrl-enter)

Копировать

```
    return goal % 5 <= small
```

Go

Editor font size %: 100

Shorter output

Expected	Run		
make_bricks(3, 1, 8) → True	True	OK	
make_bricks(3, 1, 9) → False	False	OK	
make_bricks(3, 2, 10) → True	True	OK	
make_bricks(3, 2, 8) → True	True	OK	
make_bricks(3, 2, 9) → False	False	OK	
make_bricks(6, 1, 11) → True	True	OK	
make_bricks(6, 0, 11) → False	False	OK	
make_bricks(1, 4, 11) → True	True	OK	
make_bricks(0, 3, 10) → True	True	OK	
make_bricks(1, 4, 12) → False	False	OK	
make_bricks(3, 1, 7) → True	True	OK	
make_bricks(1, 1, 7) → False	False	OK	
make_bricks(2, 1, 7) → True	True	OK	
make_bricks(7, 1, 11) → True	True	OK	
make_bricks(7, 1, 8) → True	True	OK	
make_bricks(7, 1, 13) → False	False	OK	
make_bricks(43, 1, 46) → True	True	OK	
make_bricks(40, 1, 46) → False	False	OK	
make_bricks(40, 2, 47) → True	True	OK	
make_bricks(40, 2, 50) → True	True	OK	
make_bricks(40, 2, 52) → False	False	OK	
make_bricks(22, 2, 33) → False	False	OK	
make_bricks(0, 2, 10) → True	True	OK	
make_bricks(1000000, 1000, 1000100) → True	True	OK	
make_bricks(2, 1000000, 100003) → False	False	OK	
make_bricks(20, 0, 19) → True	True	OK	
make_bricks(20, 0, 21) → False	False	OK	
make_bricks(20, 4, 51) → False	False	OK	
make_bricks(20, 4, 39) → True	True	OK	
other tests		OK	

All Correct

String-2

String-2 > xyz_there

prev | next | chance

Return True if the given string contains an appearance of "xyz" where the xyz is not directly preceeded by a period (.). So "xxyz" counts but "x.xyz" does not.

xyz_there('abcxyz') → True
xyz_there('abc.xyz') → False
xyz_there('xyz.abc') → True

Go ...Save, Compile, Run (ctrl-enter)

```
def xyz_there(str):
    if str.startswith("xyz"):
        return True

    for i in range(1, len(str) - 2):
        if str[i:i+3] == "xyz" and str[i-1] != ".":
            return True

    return False
```

Go

Expected	Run		
xyz_there('abcxyz') → True	True	OK	
xyz_there('abc.xyz') → False	False	OK	
xyz_there('xyz.abc') → True	True	OK	
xyz_there('abcxy') → False	False	OK	
xyz_there('xyz') → True	True	OK	
xyz_there('xy') → False	False	OK	
xyz_there('x') → False	False	OK	
xyz_there('') → False	False	OK	
xyz_there('abc.xyzxyz') → True	True	OK	
xyz_there('abc.xxyz') → True	True	OK	
xyz_there('.xyz') → False	False	OK	
xyz_there('12.xyz') → False	False	OK	
xyz_there('12xyz') → True	True	OK	
xyz_there('1.xyz.xyz2.xyz') → False	False	OK	
other tests		OK	

All Correct

String-2 > end_other

prev | next | chance

Given two strings, return True if either of the strings appears at the very end of the other string, ignoring upper/lower case differences (in other words, the computation should not be "case sensitive"). Note: s.lower() returns the lowercase version of a string.

end_other('Hiabc', 'abc') → True
end_other('AbC', 'HiaBc') → True
end_other('abc', 'abXabc') → True

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def end_other(a, b):
    a = a.lower()
    b = b.lower()
    return a.endswith(b) or b.endswith(a)
```

Go

Expected	Run		
end_other('Hiabc', 'abc') → True	True	OK	
end_other('AbC', 'HiaBc') → True	True	OK	
end_other('abc', 'abXabc') → True	True	OK	
end_other('Hiabc', 'abcd') → False	False	OK	
end_other('Hiabc', 'bc') → True	True	OK	
end_other('Hiabcx', 'bc') → False	False	OK	
end_other('abc', 'abc') → True	True	OK	
end_other('xyz', '12xyz') → True	True	OK	
end_other('yz', '12xz') → False	False	OK	
end_other('Z', '12xz') → True	True	OK	
end_other('12', '12') → True	True	OK	
end_other('abcXYZ', 'abcDEF') → False	False	OK	
end_other('ab', 'ab12') → False	False	OK	
end_other('ab', '12ab') → True	True	OK	
other tests		OK	

All Correct

Good job -- problem solved. You can see our solution as a

String-2 > count_code

prev | next | chance

Return the number of times that the string "code" appears anywhere in the given string, except we'll accept any letter for the 'd', so "cope" and "cooe" count.

count_code('aaacodebbb') → 1
count_code('codexxcode') → 2
count_code('cozexxcope') → 2

Go ...Save, Compile, Run (ctrl-enter)

```
def count_code(str):
    count = 0
    for i in range(len(str) - 3):
        if str[i:i+2] == "co" and str[i+3] == "e":
            count += 1
    return count
```

Go

Expected	Run		
count_code('aaacodebbb') → 1	1	OK	
count_code('codexxcode') → 2	2	OK	
count_code('cozexxcope') → 2	2	OK	
count_code('cozfxxcope') → 1	1	OK	
count_code('xxcozeyycop') → 1	1	OK	
count_code('cozcop') → 0	0	OK	
count_code('abcxyz') → 0	0	OK	
count_code('code') → 1	1	OK	
count_code('ode') → 0	0	OK	
count_code('c') → 0	0	OK	
count_code('') → 0	0	OK	
count_code('AAcodeBBcoleCCcccoreDD') → 3	3	OK	
count_code('AAcodeBBcoleCCcccorfDD') → 2	2	OK	
count_code('coAcodeBcoleccoreDD') → 3	3	OK	
other tests		OK	

All Correct

String-2 > cat_dog

prev | next | chance

Return True if the string "cat" and "dog" appear the same number of times in the given string.

cat_dog('catdog') → True
cat_dog('catcat') → False
cat_dog('1cat1cadodog') → True

Go ...Save, Compile, Run (ctrl-enter)

```
def cat_dog(str):
    return str.count('cat') == str.count('dog')
```

Go

Expected	Run		
cat_dog('catdog') → True	True	OK	
cat_dog('catcat') → False	False	OK	
cat_dog('1cat1cadodog') → True	True	OK	
cat_dog('catxxdogxxxdog') → False	False	OK	
cat_dog('catxdogxdogxcat') → True	True	OK	
cat_dog('catxdogxdogxca') → False	False	OK	
cat_dog('dogdogcat') → False	False	OK	
cat_dog('dogogcat') → True	True	OK	
cat_dog('dog') → False	False	OK	
cat_dog('cat') → False	False	OK	
cat_dog('ca') → True	True	OK	
cat_dog('c') → True	True	OK	
cat_dog('') → True	True	OK	
other tests		OK	

All Correct

String-2 > count_hi

prev | next | chance

Return the number of times that the string "hi" appears anywhere in the given string.

count_hi('abc hi ho') → 1
count_hi('ABChi hi') → 2
count_hi('hihi') → 2

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def count_hi(str):
    return str.count('hi')
```

Expected	Run	
count_hi('abc hi ho') → 1	1	OK
count_hi('ABChi hi') → 2	2	OK
count_hi('hihi') → 2	2	OK
count_hi('hiHIhi') → 2	2	OK
count_hi('') → 0	0	OK
count_hi('h') → 0	0	OK
count_hi('hi') → 1	1	OK
count_hi('Hi is no HI on ahI') → 0	0	OK
count_hi('hiho not HOHIhi') → 2	2	OK
other tests		OK

All Correct

String-2 > double_char

prev | next | chance

Given a string, return a string where for every char in the original, there are two chars.

double_char('The') → 'TThhee'
double_char('AAbb') → 'AAAAbbbb'
double_char('Hi-There') → 'HHii--TThheerree'

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def double_char(str):
  result = ""
  for i in range(len(str)):
    result += str[i] + str[i]
  return result
```

Expected	Run	
double_char('The') → 'TThhee'	'TThhee'	OK
double_char('AAbb') → 'AAAAbbbb'	'AAAAbbbb'	OK
double_char('Hi-There') → 'HHii--TThheerree'	'HHii--TThheerree'	OK
double_char('Word!') → 'WWoorrdd!!'	'WWoorrdd!!'	OK
double_char('!!') → '!!!!'	'!!!!'	OK
double_char('') → ''	''	OK
double_char('a') → 'aa'	'aa'	OK
double_char('.') → '..'	'..'	OK
double_char('aa') → 'aaaa'	'aaaa'	OK
other tests		OK

All Correct

List-2

List-2 > has22

prev | next | chance

Given an array of ints, return True if the array contains a 2 next to a 2 somewhere.

has22([1, 2, 2]) → True
has22([1, 2, 1, 2]) → False
has22([2, 1, 2]) → False

Go ...Save, Compile, Run (ctrl-enter)

```
def has22(nums):
    for i in range(len(nums) - 1):
        if nums[i] == 2 and nums[i+1] == 2:
            return True
    return False
```

Go

Expected	Run		
has22([1, 2, 2]) → True	True	OK	
has22([1, 2, 1, 2]) → False	False	OK	
has22([2, 1, 2]) → False	False	OK	
has22([2, 2, 1, 2]) → True	True	OK	
has22([1, 3, 2]) → False	False	OK	
has22([1, 3, 2, 2]) → True	True	OK	
has22([2, 3, 2, 2]) → True	True	OK	
has22([4, 2, 4, 2, 2, 5]) → True	True	OK	
has22([1, 2]) → False	False	OK	
has22([2, 2]) → True	True	OK	
has22([2]) → False	False	OK	
has22([]) → False	False	OK	
has22([3, 3, 2, 2]) → True	True	OK	
has22([5, 2, 5, 2]) → False	False	OK	
other tests		OK	

All Correct

List-2 > sum67

prev | next | chance

Return the sum of the numbers in the array, except ignore sections of numbers starting with a 6 and extending to the next 7 (every 6 will be followed by at least one 7). Return 0 for no numbers.

sum67([1, 2, 2]) → 5
sum67([1, 2, 2, 6, 99, 99, 7]) → 5
sum67([1, 1, 6, 7, 2]) → 4

Go ...Save, Compile, Run (ctrl-enter)

```
def sum67(nums):
    total = 0
    skip = False
    for n in nums:
        if n == 6:
            skip = True
            continue
        if skip:
            if n == 7:
                skip = False
            continue
        total += n
    return total
```

Go

Expected	Run		
sum67([1, 2, 2]) → 5	5	OK	
sum67([1, 2, 2, 6, 99, 99, 7]) → 5	5	OK	
sum67([1, 1, 6, 7, 2]) → 4	4	OK	
sum67([1, 6, 2, 2, 7, 1, 6, 99, 99, 7]) → 2	2	OK	
sum67([1, 6, 2, 6, 2, 7, 1, 6, 99, 99, 7]) → 2	2	OK	
sum67([2, 7, 6, 2, 6, 7, 2, 7]) → 18	18	OK	
sum67([2, 7, 6, 2, 6, 2, 7]) → 9	9	OK	
sum67([1, 6, 7, 7]) → 8	8	OK	
sum67([6, 7, 1, 6, 7, 7]) → 8	8	OK	
sum67([6, 8, 1, 6, 7]) → 0	0	OK	
sum67([]) → 0	0	OK	
sum67([6, 7, 11]) → 11	11	OK	
sum67([11, 6, 7, 11]) → 22	22	OK	
sum67([2, 2, 6, 7, 7]) → 11	11	OK	
other tests		OK	

All Correct

next | chance

List-2 > sum13

prev | next | chance

Return the sum of the numbers in the array, returning 0 for an empty array. Except the number 13 is very unlucky, so it does not count and numbers that come immediately after a 13 also do not count.

sum13([1, 2, 2, 1]) → 6
sum13([1, 1]) → 2
sum13([1, 2, 2, 1, 13]) → 6

Go ...Save, Compile, Run (ctrl-enter)

```
def sum13(nums):
    total = 0
    i = 0
    while i < len(nums):
        if nums[i] == 13:
            i += 2
            continue
        total += nums[i]
        i += 1
    return total
```

Go

Expected	Run		
sum13([1, 2, 2, 1]) → 6	6	OK	
sum13([1, 1]) → 2	2	OK	
sum13([1, 2, 2, 1, 13]) → 6	6	OK	
sum13([1, 2, 13, 2, 1, 13]) → 4	4	OK	
sum13([13, 1, 2, 13, 2, 1, 13]) → 3	3	OK	
sum13([]) → 0	0	OK	
sum13([13]) → 0	0	OK	
sum13([13, 13]) → 0	0	OK	
sum13([13, 0, 13]) → 0	0	OK	
sum13([13, 1, 13]) → 0	0	OK	
sum13([5, 7, 2]) → 14	14	OK	
sum13([5, 13, 2]) → 5	5	OK	
sum13([0]) → 0	0	OK	
sum13([13, 0]) → 0	0	OK	
other tests		OK	

All Correct

List-2 > centered_average

prev | next | chance

Return the "centered" average of an array of ints, which we'll say is the mean average of the values, except ignoring the largest and smallest values in the array. If there are multiple copies of the smallest value, ignore just one copy, and likewise for the largest value. Use int division to produce the final average. You may assume that the array is length 3 or more.

centered_average([1, 2, 3, 4, 100]) → 3
centered_average([1, 1, 5, 5, 10, 8, 7]) → 5
centered_average([-10, -4, -2, -4, -2, 0]) → -3

Go ...Save, Compile, Run (ctrl-enter)

```
def centered_average(nums):
    return (sum(nums) - max(nums) - min(nums)) // (len(nums) - 2)
```

Go

Expected	Run		
centered_average([1, 2, 3, 4, 100]) → 3	3	OK	
centered_average([1, 1, 5, 5, 10, 8, 7]) → 5	5	OK	
centered_average([-10, -4, -2, -4, -2, 0]) → -3	-3	OK	
centered_average([5, 3, 4, 6, 2]) → 4	4	OK	
centered_average([5, 3, 4, 0, 100]) → 4	4	OK	
centered_average([100, 0, 5, 3, 4]) → 4	4	OK	
centered_average([4, 0, 100]) → 4	4	OK	
centered_average([0, 2, 3, 4, 100]) → 3	3	OK	
centered_average([1, 1, 100]) → 1	1	OK	
centered_average([7, 7, 7]) → 7	7	OK	
centered_average([1, 7, 8]) → 7	7	OK	
centered_average([1, 1, 99, 99]) → 50	50	OK	
centered_average([1000, 0, 1, 99]) → 50	50	OK	
centered_average([4, 4, 4, 4, 5]) → 4	4	OK	
centered_average([4, 4, 4, 1, 5]) → 4	4	OK	
centered_average([6, 4, 8, 12, 3]) → 6	6	OK	
other tests		OK	

List-2 > big_diff

prev | next | chance

Given an array length 1 or more of ints, return the difference between the largest and smallest values in the array. Note: the built-in min(v1, v2) and max(v1, v2) functions return the smaller or larger of two values.

big_diff([10, 3, 5, 6]) → 7
big_diff([7, 2, 10, 9]) → 8
big_diff([2, 10, 7, 2]) → 8

Go ...Save, Compile, Run (ctrl-enter)

```
def big_diff(nums):
    return max(nums) - min(nums)
```

Expected	Run		
big_diff([10, 3, 5, 6]) → 7	7	OK	
big_diff([7, 2, 10, 9]) → 8	8	OK	
big_diff([2, 10, 7, 2]) → 8	8	OK	
big_diff([2, 10]) → 8	8	OK	
big_diff([10, 2]) → 8	8	OK	
big_diff([10, 0]) → 10	10	OK	
big_diff([2, 3]) → 1	1	OK	
big_diff([2, 2]) → 0	0	OK	
big_diff([2]) → 0	0	OK	
big_diff([5, 1, 6, 1, 9, 9]) → 8	8	OK	
big_diff([7, 6, 8, 5]) → 3	3	OK	
big_diff([7, 7, 6, 8, 5, 5, 6]) → 3	3	OK	
other tests		OK	

All Correct

List-2 > count_evens

prev | next | chance

Return the number of even ints in the given array. Note: the % "mod" operator computes the remainder, e.g. 5 % 2 is 1.

count_evens([2, 1, 2, 3, 4]) → 3
count_evens([2, 2, 0]) → 3
count_evens([1, 3, 5]) → 0

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def count_evens(nums):
    count = 0
    for n in nums:
        if n % 2 == 0:
            count += 1
    return count
```

Expected	Run		
count_evens([2, 1, 2, 3, 4]) → 3	3	OK	
count_evens([2, 2, 0]) → 3	3	OK	
count_evens([1, 3, 5]) → 0	0	OK	
count_evens([]) → 0	0	OK	
count_evens([11, 9, 0, 1]) → 1	1	OK	
count_evens([2, 11, 9, 0]) → 2	2	OK	
count_evens([2]) → 1	1	OK	
count_evens([2, 5, 12]) → 2	2	OK	
other tests		OK	

All Correct